Mining Input-Inherent Context: A Parameter-Free Kernel Attention Mechanism for CNNs

Yile Chen[1,2], Zeyuan Lin[1], Yawen Chen[3], Zeyi Wen[2*], Jian Chen[1*], Jin Huang[4]
[1]*South China University of Technology,* [2]*HKUST (Guangzhou),* [3]*Henan University,* [4]*South China Normal University*
{jireh.x6, zylinscut}@gmail.com, ywchen@henu.edu.cn,
wenzeyi@hkust-gz.edu.cn, ellachen@scut.edu.cn, huangjin@m.scnu.edu.cn

***Abstract*—Convolutional Neural Networks (CNNs) are widely adopted in image processing tasks due to their computational efficiency and strong inductive biases for capturing local spatial features. However, traditional CNNs inherently struggle to capture non-local statistical dependencies across high-dimensional feature spaces. While conventional attention frameworks offer a remedy, they predominantly rely on lossy heuristic pooling or parameterized sub-networks. Such mechanisms inevitably disrupt the statistical integrity of the underlying input sample distribution, thereby overlooking distribution-inherent context and suppressing fine-grained discriminative features. To address these issues, from the perspective of mining input-inherent context, we propose a theoretically grounded, parameter-free kernel attention (KA) mechanism for CNNs. Structurally rooted in the convex optimization principles of kernel machines, KA introduces a specialized warm-start initialization procedure and a training-exclusive kernel-based attention module to directly mine non-local semantic dependencies from input sample distributions. Crucially, KA is highly memory-efficient and incurs strictly zero parameter or FLOP increments during the inference phase. Extensive experiments on multiple benchmark datasets demonstrate that our approach consistently outperforms state-of-the-art baselines while faithfully preserving the native architectural simplicity and inference speed of original CNNs, maintaining a strictly linear training complexity relative to feature dimensions.**

***Index Terms*—CNNs, attention mechanism, kernel methods**

I. Introduction

Learning effective representations from high-dimensional, grid-structured data remains a foundational problem in machine learning and data mining. While recent token-based architectures such as Vision Transformers (ViTs) [1] have achieved strong performance, Convolutional Neural Networks (CNNs) [2] continue to serve as widely adopted backbone models due to their computational efficiency and strong inductive biases for modeling local spatial structures. These properties make CNNs particularly suitable for real-time and resource-constrained applications, such as autonomous driving [3] and object detection [4]. In large-scale data mining pipelines and production environments, this structural and computational predictability translates directly to higher throughput, lower carbon footprints, and deterministic latency bounds across diverse edge devices.

However, standard convolutional operators are fundamentally limited by local receptive fields [5], [6], which restrict their ability to capture long-range dependencies and global contextual interactions in feature representations. To address this limitation, attention mechanisms have been widely integrated into CNN architectures [7]–[9]. While effective, these methods often introduce additional parameters or permanent architectural modifications, which inevitably increases computational costs and deployment complexity. In addition, many existing designs rely on heuristic feature aggregation strategies over intermediate feature maps, which may reduce spatial information and limit the modeling of fine-grained discriminative features. For instance, conventional global average pooling operations collapse entire spatial dimensions into fixed-size vectors, a process that inherently discards critical topological relationships and statistical variances necessary for subtle pattern discovery.

From a data mining perspective, a key challenge is how to design efficient attention mechanisms that can better exploit feature interactions while maintaining the simplicity of CNN architectures. Recent studies have shown that dot-product attention can be interpreted as a form of similarity computation in kernel-induced feature spaces [10], establishing connections between attention mechanisms and kernel methods. Motivated by this observation, several approaches have explored kernel-based formulations to improve the efficiency and scalability of attention mechanisms [11]. However, integrating kernel-based modeling into CNNs remains challenging, as existing methods often incur high computational overhead or alter network structures, undermining the efficiency of standard CNN backbones.

To this end, we propose a Kernel Attention (KA) mechanism for CNNs from the perspective of kernel-based representation learning. KA is designed as a lightweight training-time module that enhances feature interaction modeling through kernel-based similarity estimation. Crucially, a warm-start initialization strategy is introduced to stabilize optimization and accelerate convergence. This effectively bypasses the gradient stagnation and instability typically arising from the alignment between non-parametric methods and parametric networks. Importantly, KA is fully compatible with standard CNN architectures and does not modify the inference pipeline. As a result, it introduces no additional parameters or computational cost during inference, while ensuring a highly efficient and stable training process.

To evaluate the practical efficacy of our proposed frame-

*Corresponding authors.

work, we conduct comprehensive evaluations across multiple large-scale benchmarks representing varied data mining scenarios. The empirical outcomes confirm that KA successfully infuses non-local awareness into standard convolutional layers, demonstrating its robust viability for production-grade, latency-sensitive applications where deployment budgets are strictly constrained.

To summarize, the main contributions of this paper are as follows:

- We propose Kernel Attention (KA), bridging statistical kernel methods and deep learning to capture inherent feature interactions without changing the standard CNN inference architecture.
- We introduce a training-only kernel attention module with warm-start initialization, enabling parameter- and FLOP-free inference with stable training.
- We validate KA against state-of-the-art attention methods on standard benchmarks. KA improves accuracy while preserving the inference latency of vanilla CNNs.

II. Related Work

This section reviews prior literature closely aligned with our work, which centers around the intersection of statistical kernel learning and deep architectures. Concurrently, we contextualize the foundational gaps that motivate our study from a structured data-mining perspective.

A. Kernel Learning and Context Mining in CNNs

Integrating statistical kernel methods into convolutional neural networks (CNNs) has been widely explored to enhance representation capacity and structural interpretability. For large-scale data mining, early frameworks leveraged random features [12] or unified deep networks with kernel machines [13]. However, when applied to convolutional backbones, conventional efforts primarily focus on the explicit "kernelization" of internal operators or hidden tensors. For instance, Convolutional Kernel Networks (CKNs) [14], [15] replace standard convolutions with kernel mappings in a Reproducing Kernel Hilbert Space (RKHS). To capture higher-order non-linear interactions, Kernel Pooling [16] and Compact Bilinear Pooling [17] employ kernel-driven bilinear feature aggregations via techniques like Tensor Sketch. Although classic kernel algorithms like the Nyström method have recently been used to linearize attention mechanisms [18], seamlessly extending such formulations to standard CNN architectures without introducing structural or computational overhead remains an open challenge.

Unlike architectures that modify internal feature dimensions or convolutional operators, thereby introducing optimization friction and scaling bottlenecks, our KA framework adopts an alternative formulation to bypass permanent structural changes. Instead of kernelizing network parameters or filtering steps, KA draws motivation from optimization perspectives to extract non-parametric, data-dependent attention weights. By directly mining contextual dependencies from raw sample distributions, our approach establishes a parameter-free kernel attention mechanism that faithfully preserves the structural simplicity and inference efficiency of the base CNN.

B. Attention Mechanisms for CNNs

Attention mechanisms have become a fundamental component for improving convolutional neural networks (CNNs) by enabling adaptive feature recalibration and enhanced representation learning [19], [20]. Existing methods can be broadly categorized into two main paradigms: dimension-wise feature recalibration and global relational modeling.

Dimension-wise feature recalibration methods focus on reweighting channel or spatial responses through aggregated statistical descriptors. The Squeeze-and-Excitation (SE) network [7] first introduced channel-wise attention by leveraging global average pooling to model inter-channel dependencies. Subsequently, the Bottleneck Attention Module (BAM) [21] and the Convolutional Block Attention Module (CBAM) [8] extended this idea by incorporating both average and max pooling operations to jointly capture complementary spatial statistics. Further extensions explore more diverse design strategies: Selective Kernel Networks (SKNet) [22] and ResNeSt [23] introduce dynamic receptive field selection via grouped attention mechanisms; ECANet [24] improves efficiency by replacing fully connected layers with lightweight 1D convolutions; FcaNet [25] incorporates frequency-domain representations using discrete cosine transforms; and EPSA [26] integrates multi-scale feature aggregation into channel recalibration.

Global relational modeling methods aim to model long-range dependencies through global interactions between feature elements. Non-local Neural Networks (NL) [27] compute pairwise affinities across all spatial positions. Building upon this formulation, the Double Attention Network (A^2Net) [28] decouples attention into feature gathering and distribution stages, while Global Second-Order Pooling (GSoP) [9] leverages covariance-based representations to capture higher-order channel correlations. To improve computational efficiency, GCNet [29] simplifies non-local operations by combining global context modeling with channel-wise recalibration. In addition, SimAM [30] provides a parameter-free formulation by optimizing a neuron-level energy function, offering an alternative perspective on attention design. However, these methods typically rely on heavy parametric transformations or localized computations, introducing a trade-off between computational overhead and representation capacity. Balancing global feature interactions with inference efficiency within standard convolutional structures remains an active area of research.While effective, these approaches generally rely on heuristic aggregation functions or parameterized modules, which may introduce information compression effects during feature recalibration.

III. Preliminaries

A. Kernel Machines

The kernel machine maps a non-linear problem into a feature space where the problem may be linearly separa-

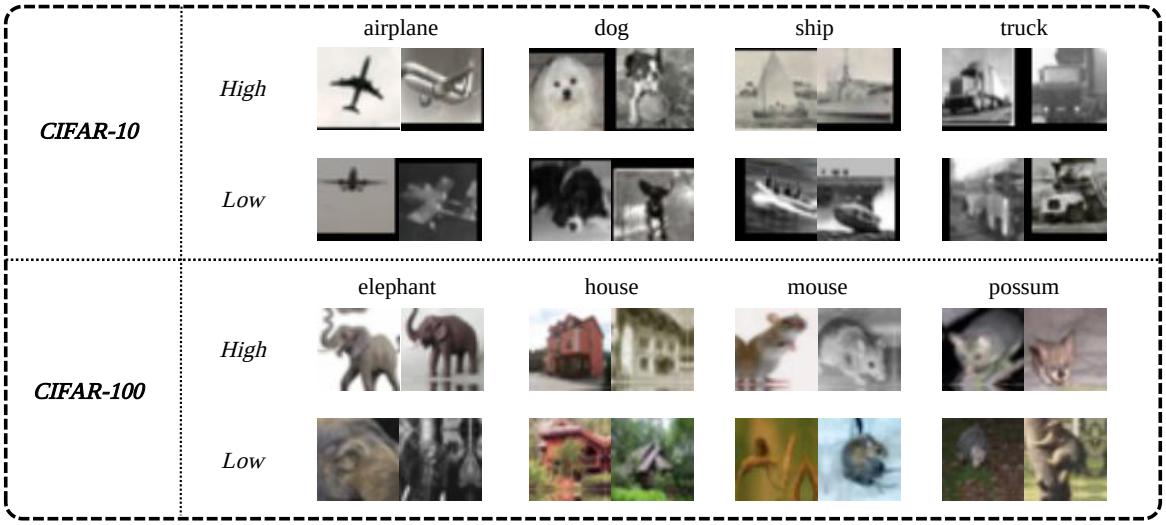


Fig. 1: Comparison of images with significantly different attention scores under the identical class label from the *CIFAR-10/CIFAR-100*. Four categories are randomly selected from both *CIFAR-10* and *CIFAR-100* for visualization. KA computes the attention scores, with High and Low denoting the highest- and lowest-scoring samples in each class.

ble [31], [32]. Formally, suppose we have a training dataset $\{X, \boldsymbol{y}\}$ with n instances, where $\{X \in \mathbb{R}^{n\times d}, \boldsymbol{y} \in \mathbb{R}^n\} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), ..., (\boldsymbol{x}_n, y_n)\}$, and $(\boldsymbol{x}_i, y_i)$ denotes the instance $\boldsymbol{x}_i \in \mathbb{R}^d$ with its label y_i. The goal of training a kernel machine is to find the optimal weight vector $\boldsymbol{\omega}$ that minimizes the structural risk as follows.

$$\arg\min_{\boldsymbol{\omega}} L(\boldsymbol{\omega}) = \arg\min_{\boldsymbol{\omega}} \left(\frac{1}{n} \sum_{i=1}^{n} l\big(f(\boldsymbol{\omega}, \boldsymbol{x}_i), y_i\big) + \frac{\lambda}{2}\|\boldsymbol{\omega}\|^2 \right), \tag{1}$$

where λ denotes the regularization constant. The decision function is computed as $f(\boldsymbol{\omega}, \boldsymbol{x}_i) = \langle \boldsymbol{\omega}, \phi(\boldsymbol{x}_i) \rangle$ where $\boldsymbol{\omega}$ is defined on the *reproducing kernel Hilbert space* (RKHS) and $\langle \cdot, \cdot \rangle$ is the inner product on RKHS. The function $\phi(\cdot)$ maps the instances from their original data space to a higher dimensional feature space induced by the kernel function. Since Equation (1) falls under the *representer theorem* [33] and involves convex optimization, we can derive that a minimizer of Equation (1) is $\boldsymbol{\omega} = \sum_{j=1}^{n} \alpha_j \phi(\boldsymbol{x}_j)$. Based on the *reproducing property* [34], we have

$$f(\boldsymbol{\omega}, \boldsymbol{x}_i) = \sum_{j=1}^{n} \alpha_j k(\boldsymbol{x}_i, \boldsymbol{x}_j),$$

where $k(\boldsymbol{x}_i, \boldsymbol{x}_j) = \langle \phi(\boldsymbol{x}_i), \phi(\boldsymbol{x}_j) \rangle$ is a positive-definite kernel. By substituting the expressions of $f(\boldsymbol{\omega}, \boldsymbol{x}_i)$ and $\boldsymbol{\omega}$ into Equation (1), we reformulate the objective function in terms of $\boldsymbol{\alpha}$ as follows,

$$\arg\min_{\boldsymbol{\alpha}} L(\boldsymbol{\alpha}) = \arg\min_{\boldsymbol{\alpha}} \left(\frac{1}{n} \sum_{i=1}^{n} l\Big(\sum_{j=1}^{n} \alpha_j k(\boldsymbol{x}_i, \boldsymbol{x}_j), y_i\Big) + \frac{\lambda}{2} \Big\| \sum_{j=1}^{n} \alpha_j \phi(\boldsymbol{x}_j) \Big\|^2 \right),$$

where $\boldsymbol{\alpha} = [\alpha_1 \ \dots \ \alpha_n]^T$ is an n-dimension vector, each dimension of which corresponds to the contribution of a training instance to the kernel machine.

B. Attentions from Kernel Machines

Attention mechanisms and kernel machines, two foundational paradigms in deep learning and traditional machine learning, have been shown to exhibit similar properties and are equivalent under certain conditions [35]. Recently, the most widely used form of the attention mechanism is the Scaled Dot-Product Attention [36], defined as follows.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \tag{2}$$
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where $\boldsymbol{X} := [x_1 \ \dots \ x_n]^T \in \mathbb{R}^{n\times D_x}$ is a given input sequence of n feature vectors; W_Q, $W_K \in \mathbb{R}^{d_k \times D_x}$, and $W_V \in \mathbb{D}^{D_v \times D_x}$ are the weight matrices. We denote $Q := [q_1 \ \dots \ q_n]^T$, $K := [k_1 \ \dots \ k_n]^T$, and $V := [v_1 \ \dots \ v_n]^T$, where the vector q_i, k_i, v_i for $i = 1, \ \dots \ , n$ are the query, key, and value vectors, respectively.

Meanwhile, the above attention mechanism can be expressed as the kernel methods following the work [37] where

$$f(\boldsymbol{x}) = \sum_{i=1}^{N} \frac{\Phi(\boldsymbol{x})^\top \Phi(\boldsymbol{x}_i)}{h(\boldsymbol{x})} \boldsymbol{v}_i + b = \text{attention},$$
$$\boldsymbol{v}_i = \frac{\alpha_i}{h(\boldsymbol{x}_i)}, \quad h(\boldsymbol{x}) = \sum_{i=1}^{N}\sum_{j=1}^{N} \Phi(\boldsymbol{x})^\top \Phi(\boldsymbol{x}_i). \tag{3}$$

The function $\Phi(\cdot)$ maps the input $\boldsymbol{x}$ into another space. To make sure that the attention in Equation (3) is equivalent to the kernel machines as in Equation (4), the inner product of $\Phi(\cdot)^\top$ and $\Phi(\cdot)$ has to satisfy the condition below.

$$\Phi(\boldsymbol{x})^\top \Phi(\boldsymbol{x}_i) = \sum_{t=0}^{\infty} \sum_{n_1+\cdots+n_{d_k}=t} \phi(\boldsymbol{x})\phi(\boldsymbol{x}_i), \tag{4}$$
$$\phi(\boldsymbol{x}) = \frac{\left(\frac{\boldsymbol{x}_1}{\sqrt[4]{d_k}}\right)^{n_1} \cdots \left(\frac{\boldsymbol{x}_{d_k}}{\sqrt[4]{d_k}}\right)^{n_{d_k}}}{\sqrt{n_1! \ldots n_{d_k}!}},$$
$$\phi(\boldsymbol{x}_i) = \frac{\left(\frac{\boldsymbol{x}_{i1}}{\sqrt[4]{d_k}}\right)^{n_1} \cdots \left(\frac{\boldsymbol{x}_{id_k}}{\sqrt[4]{d_k}}\right)^{n_{d_k}}}{\sqrt{n_1! \ldots n_{d_k}!}}.$$

IV. Methodology

To address the limitations of current CNN attention methods, we propose a parameter-free kernel attention (KA) mechanism for CNNs, which leverages convex information from kernel machines to capture global attention in images. As the workflow in Figure 2, KA comprises two main components: a specialized initialization procedure (indicated by the blue arrows) and a kernel-based attention module (indicated by the orange arrows). In the following section, we first present the theoretical motivation behind KA in conjunction with the preliminaries. After that, we describe the details and overall process of KA, and conclude by discussing its advantages compared to existing methods.

A. Theoretical Motivation

As shown in Section III-B, the attention operation in Equation (2) can be reformulated as a kernel machine as in Equation (3). The term $\boldsymbol{v}_i$ is computed using the weight $\boldsymbol{\alpha}$ of the kernel machine. For the coefficient $\frac{\Phi(\boldsymbol{x})\Phi(\boldsymbol{x}_i)}{h(\boldsymbol{x})}$, the unknown term is the inner product of $\Phi(\boldsymbol{x})$ and $\Phi(\boldsymbol{x}_i)$. In our work, instead of finding or learning such coefficients that satisfy the conditions in Equation (4), we utilize the convolution multiplication inherited in CNNs to approximate those coefficients, as both kernel machines and CNNs aim to learn an appropriate mapping of the input. Moreover, we further plug the weight $\boldsymbol{\alpha}$ from the kernel machine into CNNs to simulate the term $\boldsymbol{v}_i$. With the modified convolutional filter and the weight $\boldsymbol{\alpha}$ to simulate Equation (4), our method can enhance the CNN performance with lightweight and user-friendly attention extracted from the kernel machines. The implementation details are as follows.

Algorithm 1: Execution Pipeline of Kernel Attention

Input : Dataset $\mathcal{D} = \{(\boldsymbol{X}_i, y_i)\}_{i=1}^m$, Base CNN layers $\{1, \ldots, L\}$, Kernel $K(\cdot,\cdot)$, Regularizer R, Learning rate η.

Output: Optimized network weights with convex attention tracking.

// Phase 1: Warm-Start Node Initialization

$F^{(l-1)} \leftarrow \text{RandomSample}(F^{(l-1)}, n_k)$;

for $i \leftarrow 1$ **to** n_k **do**

  Initialize filter weights $C_i^{(l)}$ via Eq. (5);

// Phase 2: Convex Optimization via SMO

Partition feature map $F^{(l-1)}$ along channel dimensions;

$\boldsymbol{\alpha}, b \leftarrow \text{SMO-Solver}(F^{(l-1)}, y, K, R)$;

// Phase 3: Runtime Forward Propagation with KA

foreach *training iteration* **do**

  Update $F_{j,*}^{(l-1)} \leftarrow \sum_{p=1}^{P} \alpha_j F_{j,p}^{(l-1)}$ via Eq. (6) ;
  // Apply convex scaling

  Forward propagate through hidden layers $l \in \{2, \ldots, L\}$;

  Compute loss, backward propagate, and update parameters via SGD(η);

B. KA in CNN Training

We present the proposed KA mechanism and detail its integration into modern CNN architectures. We begin by introducing the two core components of KA: the initialization procedure and the kernel-based attention module. We then provide a summary of the overall workflow of KA.

Initialization procedure. KA introduces a warm-start initialization method for convolutional filters, illustrated by the blue arrows in Figure 2. As shown in Equation (2), the matrices Q and K can be interpreted as the original input mapped into a higher-dimensional feature space by multiplying with matrices W_Q and W_K. In CNNs, the input to the convolutional layer can be either an image or a feature map. Therefore, both the image and the feature map can be treated as the mapped input Q in the attention mechanism. The matrix K corresponds to the convolution filter C. However, C is typically initialized randomly or using other techniques, and unlike in the attention technique, it is not directly related to the input. To better simulate the attention operation, we propose the following initialization strategy, which establishes a map from the input to the convolution filter.

$$C_{i,j}^{(l)} = W \frac{1}{m} \sum_{k=1}^{m} F_j^{(l-1)} W', \quad j \in \{1, \ldots, n^{l-1}\}, \tag{5}$$

where n^{l-1} is the number of channels in the feature map outputted by the $(l-1)$-th layer. Let $C_{i,j}^{(l)} \in \mathbb{R}^{n^l \times h \times w}$ denote the j-th channel of the i-th convolution kernel in the l-th layer and $F_j^{(l-1)} \in \mathbb{R}^{n^l \times d_h \times d_w}$ is the j-th channel of the output feature map of the $(l-1)$-th layer. We use the projection

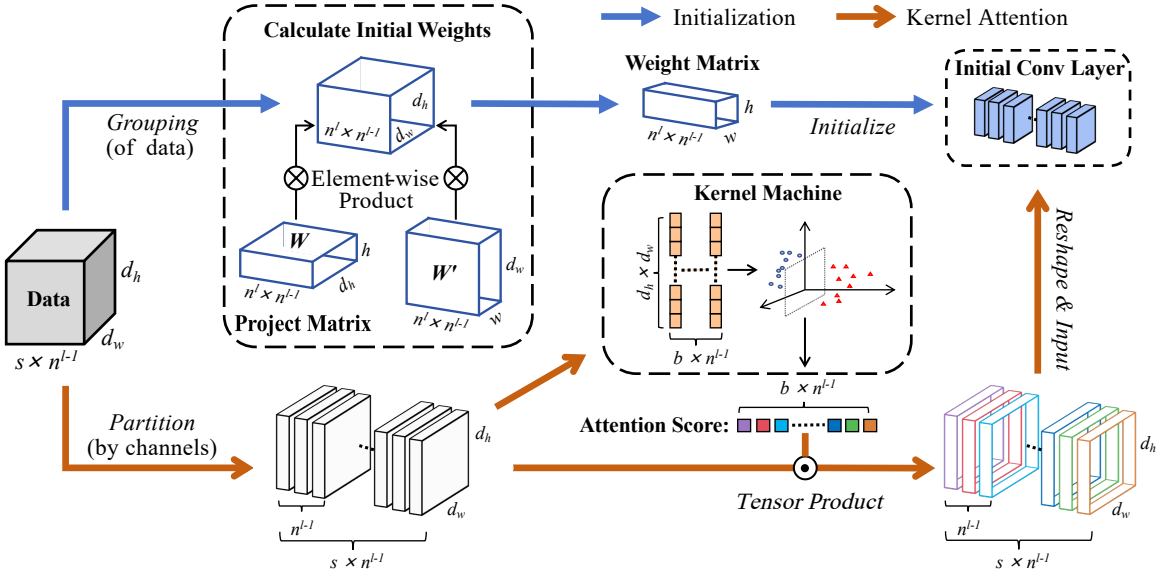


Fig. 2: **The overview of KA.** This module consists of two parts: Initialization and Kernel attention.

matrix $W \in \mathbb{R}^{n^l \times h \times d_h}$ and $W' \in \mathbb{R}^{n^l \times d_w \times w}$ to transform the information in feature map $F_j^{(l-1)}$ into the convolution filter. We do not store and update the matrix W once the initialization is finished. Thus no extra parameters are brought into the original CNN architectures. The corresponding pseudocode for this warm-start initialization procedure is detailed in Phase 1 of Algorithm 1.

Attention from kernel machines. With the initialization, we leverage the convex attention information from kernel machines to boost CNN training. This attention module, highlighted by the orange arrows in Figure 2, works with the warm-start initialization to form the attention mechanism described in Equation (2).

The algorithmic optimization process for evaluating and computing these kernel-driven multipliers is integrated into Phase 2 of Algorithm 1. Kernel Attention first partitions each mini-batch along the channel dimension to reconstruct a training set suitable for SVM learning. Then, based on the principles of the SMO algorithm, the Lagrangian multipliers $\boldsymbol{\alpha}$ corresponding to the samples are efficiently computed and stored for use during CNN training. The $\boldsymbol{\alpha}$ in the kernel machine represents the priority and significance of the corresponding input as stated in Section III. Thus, we integrate these weights with convex information into CNN to better capture the attention. During the training phase, we multiply the input feature map by α as follows.

$$F_j^{(l)} = \alpha_j \sum_p^{\mathrm{P}} F_{j,p}^{(l-1)} * C_{i,j}^{(l)}, \quad j \in \{1, \ldots, n^{l-1}\}, \tag{6}$$

where $F_{j,p}^{(l-1)}$ is the p-th patch in $F_j^{(l-1)}$. By sliding the window with size $h \times w$ on the input with a specified stride, we have the total number P of the patches used in the convolutional kernel. Unlike existing methods that require iterative updates of the attention matrix during training, Kernel Attention computes it once, greatly reducing training cost. Compared to approaches that modify network structures, it is more lightweight and can be further accelerated via GPU parallelism alongside CNN training. Importantly, $\boldsymbol{\alpha}$ is typically sparse, leading to sparse training in CNNs. These weights are only used during training and do not add any extra parameters to the final model.

Overview of KA. Overview of KA. After introducing the two key components of KA, we now provide an overview of the entire KA workflow, with the unified system execution pipeline systematically delineated in Algorithm 1. KA first begins with an initialization step before the training phase. Specifically, this step involves randomly selecting input samples corresponding to the number of output channels in the network's initial convolution layer. Using the initialization strategy, the initial weights of the convolution layer are determined, establishing a connection between these weights and the input samples. After that, KA incorporates the attention from kernel machines into CNNs to help extract global attention from the input samples. In particular, the input samples are partitioned across the channel domain to formulate a surrogate training set for the kernel machine, which then derives the corresponding Lagrange multiplier vector $\boldsymbol{\alpha}$. During the training phase, the input feature map is multiplied by the corresponding α values.

C. Advantages of KA

Compared to existing attention mechanisms in CNNs, the proposed KA exhibits four distinctive advantages.

First, KA derives attention information directly from raw training samples during optimization. While most methods estimate importance from intermediate features, recent studies [38], [39] show that raw-sample information provides complementary supervision. Extracting cues at the data level helps the network capture informative patterns, facilitating more discriminative representation learning.

TABLE I: Comparison of attention methods in design, parameterization, focus, and complexity. Operators include: spatial/channel average pooling (SAP/CAP), spatial/channel max pooling (SMP/CMP), standard/channel-wise convolution (Conv2D/Conv1D), and fully connected layers (FC). Notation: k: number of filters, r: reduction ratio, d_h, d_w: feature map dimensions, n_c: channels, l: baseline depth.

Methods	Operators	Parameters	Focus	Complexity
SE [7]	SAP, FC, ReLU	$2n_c^2/r^2$	Hidden feature	$\mathcal{O}(n_c^2 l/r)$
CBAM [8]	SAP, SMP, FC, ReLU, CAP, CMP, BN, Conv2D	$2n_c^2/r^2 + 2k^2$	Hidden feature	$\mathcal{O}(n_c^2 l/r)$
ECA [24]	GAP, Conv1D	k	Hidden feature	$\mathcal{O}(d_h d_w l n_c)$
SimAM [30]	GAP, /, $\odot$, +	0	Hidden feature	$\mathcal{O}(d_h d_w l n_c)$
Ours (KA)	$\odot$	**0**	**Input feature**	$\boldsymbol{\mathcal{O}(d_h d_w n_c)}$

Second, KA formulates attention extraction as a principled optimization problem without architectural modifications. Unlike mainstream mechanisms requiring extra structural components (Table I), KA operates entirely within the training procedure. It introduces no trainable parameters, preserves the original network, and maintains standard asymptotic computational complexity.

Third, KA provides a theoretically grounded and interpretable formulation based on kernel theory. Unlike heuristic modules that are difficult to analyze uniformly, KA derives attention weights through a principled kernel framework, improving both interpretability and controllability.

Fourth, KA is largely architecture-independent, facilitating its seamless application beyond convolutional models. Unlike mechanisms tightly coupled with spatial structures, KA utilizes coordinate-free kernel mappings and optimization objectives. This eliminates dependence on specific tensor geometries, ensuring favorable cross-architecture compatibility with both CNNs and Transformers.

V. Experimental Studies

To rigorously validate our proposed KA, our experimental protocol spans two distinct dimensions: cross-domain image classification and the downstream fine-tuning of pretrained models. We benchmark KA against both vanilla backbones and current state-of-the-art attention paradigms to evaluate its efficacy as a plug-and-play architectural enhancement. For rigor and reproducibility, all evaluations are executed under a strictly standardized training and optimization protocol. The corresponding code for our method can be found in the respective repository on GitHub https://github.com/Jireh-Chen/KernalAttention.

A. Datasets and Experimental Setup

Datasets and Metrics. We evaluate our proposed method across six diverse benchmarking datasets spanning multiple domains: (1) *Muffin-vs-Chihuahua (Muffin)* [40] and (2) *Pizza* [41] for fine-grained food classification; (3) *S-cov* [42] consisting of RGB CT scans for medical virus detection; (4) *CIFAR-10* [43] and (5) *CIFAR-100* [43] for general object recognition; and (6) *SVHN* [44] for street view house number digit recognition. Table II summarizes the detailed statistics. We adopt Top-1 Accuracy (%) as the primary evaluation metric, supplemented by the F_1-score to handle potential class imbalances.

TABLE II: Details of the Image Classification Benchmarks utilized in our evaluation.

Dataset	# Training Instances	# Test Instances	# Classes
Muffin	4,733	1,184	2
Pizza	1,600	366	2
S-cov	1,998	483	2
CIFAR-10	50,000	10,000	10
CIFAR-100	50,000	10,000	100
SVHN	73,257	26,032	10

Baseline methods. In our experimental studies, we compare our method with existing advanced methods, which are detailed as follows.

- **CBAM** [8]: CBAM enhances the model's perceptual capabilities by integrating channel and spatial attention mechanisms within CNNs.
- **ECA** [24]: ECA captures channel attention efficiently through a one-dimensional convolution, with the kernel size adaptively determined by a nonlinear mapping of the channel dimension.
- **SE** [7]: SE learns the relative importance of each feature channel, enhancing relevant features while suppressing those less pertinent to the current task.
- **SimAM** [30]: SimAM calculates three-dimensional attention weights for feature maps by optimizing an energy function.

Experimental setup. All experiments are executed on a Linux server equipped with an Intel Xeon E5-2678 v3 CPU, 128 GB RAM, and two NVIDIA GeForce RTX 2080 Ti GPUs. We employ well-established network architectures as base models for evaluation, including ResNet-18, ResNet-34, ResNet-50, and ViT-B. Images undergo uniform pre-processing during training, and all compared methods perform inference on native, unmodified samples. Detailed implementation parameters and hyperparameter configurations are specified below.

- Baseline modules leverage publicly available official implementations operating under their default configurations.
- For our KA module, the Radial Basis Function (RBF) kernel, $k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|_2^2)$, is adopted to map the feature space, where the regularization penalty $C = 1/\lambda$ is selected from $[2^0, 2^3]$ and the kernel parameter γ ranges within $[10^{-4}, 1]$.
- The initial learning rate is grid-searched from $\{10^{-1}, 10^{-2}\}$, with a learning rate decay scheduled every 80 epochs over

TABLE III: Comprehensive performance evaluation: F_1-score (%) and Test Accuracy (%) of different networks with various attention modules on six datasets. Bold indicates the best result within each backbone group.

Method	Muffin		Pizza		S-cov		CIFAR-10		CIFAR-100		SVHN		Avg.	
	F_1	Acc.	F_1	Acc.	F_1	Acc.	F_1	Acc.	F_1	Acc.	F_1	Acc.	F_1	Acc.
ResNet-18	88.74	89.02	76.23	77.32	65.12	67.91	92.74	92.74	70.87	71.00	95.63	95.76	81.56	82.29
+CBAM	88.04	88.68	74.59	76.23	69.46	71.43	92.35	92.36	70.16	70.24	95.82	96.01	81.74$^{\uparrow 0.18}$	82.49$^{\uparrow 0.20}$
+ECA	88.81	89.19	76.50	76.50	70.75	72.67	92.23	92.22	68.43	68.53	95.65	95.83	82.06$^{\uparrow 0.50}$	82.49$^{\uparrow 0.20}$
+SE	88.73	88.85	77.05	78.68	68.21	71.64	92.59	92.59	71.41	71.52	95.80	95.96	82.30$^{\uparrow 0.74}$	83.21$^{\uparrow 0.92}$
+SimAM	87.96	88.43	75.11	75.96	70.80	72.46	92.79	92.79	71.62	71.79	95.85	95.99	82.36$^{\uparrow 0.80}$	82.90$^{\uparrow 0.61}$
+Ours	**89.81**	**89.78**	**77.60**	**78.83**	**70.90**	**73.50**	**92.91**	**92.92**	**71.87**	**71.94**	**95.95**	**96.12**	**83.17**$^{\uparrow \mathbf{1.61}}$	**83.76**$^{\uparrow \mathbf{1.47}}$
ResNet-34	87.97	89.70	75.62	78.14	64.43	67.08	92.46	92.47	71.01	71.11	95.86	96.01	81.23	82.42
+CBAM	88.30	89.44	76.20	77.32	69.39	71.22	92.70	92.72	70.18	70.38	**96.20**	**96.31**	82.16$^{\uparrow 0.93}$	82.90$^{\uparrow 0.48}$
+ECA	87.55	88.60	77.32	77.87	72.05	74.53	92.83	92.84	69.45	69.57	96.09	96.22	82.55$^{\uparrow 1.32}$	83.27$^{\uparrow 0.85}$
+SE	88.05	88.26	73.49	76.23	67.97	69.77	92.60	92.61	71.08	71.21	96.03	96.18	81.54$^{\uparrow 0.31}$	82.38$^{\downarrow 0.04}$
+SimAM	87.12	88.60	76.20	77.87	70.13	72.26	92.98	92.99	71.11	71.21	95.96	96.13	82.25$^{\uparrow 1.02}$	83.18$^{\uparrow 0.76}$
+Ours	**88.88**	**89.95**	**78.95**	**79.24**	**72.29**	**74.95**	**93.02**	**93.03**	**71.19**	**71.32**	96.07	96.27	**83.40**$^{\uparrow \mathbf{2.17}}$	**84.13**$^{\uparrow \mathbf{1.71}}$
ResNet-50	83.07	83.62	70.76	70.49	71.15	73.29	92.28	92.29	70.21	70.36	96.07	96.18	80.59	81.04
+CBAM	**86.42**	**87.33**	**74.32**	74.04	71.09	71.02	92.13	92.13	66.86	67.03	96.10	96.25	81.15$^{\uparrow 0.56}$	81.30$^{\uparrow 0.26}$
+ECA	84.27	84.29	72.68	73.77	70.07	73.09	92.68	92.68	69.96	70.10	96.06	96.20	80.95$^{\uparrow 0.36}$	81.69$^{\uparrow 0.65}$
+SE	85.95	86.40	74.30	**75.68**	71.28	74.95	92.48	92.48	70.24	70.32	96.05	96.19	81.72$^{\uparrow 1.13}$	82.67$^{\uparrow 1.63}$
+SimAM	81.89	81.93	68.58	68.58	69.02	70.60	92.60	92.61	71.48	71.62	95.93	96.11	79.92$^{\downarrow 0.67}$	80.24$^{\downarrow 0.80}$
+Ours	84.15	85.39	72.94	74.32	**73.58**	**75.98**	**92.81**	**92.83**	**72.08**	**72.20**	**96.14**	**96.32**	**81.95**$^{\uparrow \mathbf{1.36}}$	**82.84**$^{\uparrow \mathbf{1.80}}$
ViT-B (MLP)	99.49	99.66	98.09	98.63	70.90	75.78	94.29	94.32	75.15	87.44	93.68	93.91	88.60	91.62
+Ours	**99.57**	**99.75**	**98.19**	**98.91**	**73.63**	**77.43**	**94.40**	**94.39**	**75.24**	**87.53**	**94.37**	**94.79**	**89.23**$^{\uparrow \mathbf{0.63}}$	**92.13**$^{\uparrow \mathbf{0.51}}$

a total 100-epoch training span.

- The mini-batch size is tuned within the candidate set of $\{32, 64\}$.
- Training early-terminates if the absolute variance of loss falls below 10^{-4} across three consecutive epochs, or upon reaching the maximum 100-epoch limit.
- To mitigate experimental variance and ensure statistical stability, all individual trials are independently executed three times, with the mean performance metrics reported. Optimal model checkpoint selection for each run is guided by peak validation accuracy.
- All baseline methods and our proposed KA framework are thoroughly developed and evaluated within the PyTorch platform.

B. Comparison with Current Methods

To systematically validate the empirical efficacy, generalization resilience, and architectural compatibility of our proposed KA, this subsection presents a comprehensive comparative evaluation against prominent state-of-the-art plug-and-play attention mechanisms, including SE, CBAM, ECA, and SimAM. Our benchmark analysis is structured across three complementary dimensions: (i) performance scaling and robust representation across fluctuating convolutional network depths (ResNet-18/34/50); (ii) rigorous non-parametric statistical validation to confirm empirical significance; and (iii) extension to heterogeneous, attention-native architectures (ViT-B) to verify cross-paradigm architectural synergy.

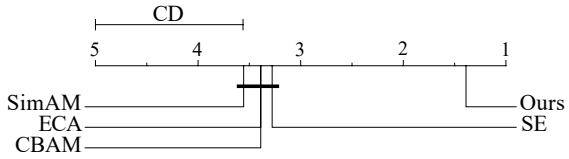


Fig. 3: Comparison of all attention methods against each other using the Nemenyi test. Groups of methods that are not significantly different are connected.

Performance Across ConvNet Depths. Table III presents a quantitative evaluation of KA against state-of-the-art attention mechanisms across various network depths. Evaluated on the lightweight ResNet-18 architecture, KA secures a clear advantage over all baselines, boosting the average test accuracy to 83.76% with a 1.47% net gain. This performance margin amplifies on challenging classification tasks. On medical image detection (*S-cov*), for instance, KA advances the ResNet-34 baseline from 67.08% to 74.95% (a 7.87% absolute improvement) while elevating the F_1-score by 7.86%. Scaling to the deeper ResNet-50 backbone further validates the robustness of our method. Although heavy parameterized modules like CBAM and SE capture localized representation advantages on selected datasets, such as *Muffin* and *Pizza*, KA secures the optimal macro-average test accuracy of 82.84% (outperforming the vanilla baseline by 1.80%) alongside an average F_1-score of 81.95%. Such generalized efficacy implies that our architecture-preserving training scheme effectively

TABLE IV: Comparison of inference speed and parameter efficiency of various attention modules across different networks. Bold indicates the best performance.

Method	Parameters	△ Parameters	FLOPs	Speed (FPS)
ResNet-18	10.65 M	**0**	0.54 G	**241.27**
+CBAM	10.74 M	0.08 M		105.14
+ECA	10.65 M	24		171.96
+SE	10.74 M	0.08 M		160.76
+SimAM	10.65 M	**0**		162.14
+Ours	**10.65 M**	**0**		**241.27**
ResNet-34	20.29 M	**0**	1.16 G	**137.11**
+CBAM	20.45 M	0.16 M		55.08
+ECA	20.29 M	48		95.35
+SE	20.45 M	0.15 M		87.12
+SimAM	20.29 M	**0**		87.51
+Ours	**20.29 M**	**0**		**137.11**
ResNet-50	22.42 M	**0**	1.31 G	**95.64**
+CBAM	24.85 M	2.43 M		45.85
+ECA	22.42 M	48		70.52
+SE	24.85 M	2.43 M		68.19
+SimAM	22.42 M	**0**		66.71
+Ours	**22.42 M**	**0**		**95.64**

bypasses the optimization pathologies commonly triggered by parameter scaling in deep networks.

Statistical Significance. To evaluate the statistical significance of these empirical differences, we perform a non-parametric Friedman test followed by Nemenyi post-hoc analyses at a significance level of $p = 0.05$. As illustrated by the critical difference (CD) diagram in Figure 3, the central tendencies of KA exhibit a statistically significant margin over alternative attention mechanisms, confirming its robust empirical advantages under rigorous statistical controls.

Heterogeneous Architectural Generalization. To evaluate architectural robustness, we extend KA from CNNs to the attention-native Vision Transformer (ViT-B). While mainstream CNN attention modules (e.g., SE, CBAM, and ECA) are structurally coupled with 2D spatial features—rendering them mathematically incompatible with 1D token sequences without altering the multi-head self-attention (MHSA) blocks—KA seamlessly integrates into the ViT backbone due to its architecture-preserving kernel formulations. As shown in Table III, KA delivers consistent performance improvements over the vanilla ViT across all benchmarks. Notably, on the *S-cov* dataset, it improves the baseline by 2.73% in F_1-score and 1.65% in test accuracy. These consistent advancements on near-saturated datasets validate that our approach possesses a universal architectural synergy transcending conventional convolutional boundaries.

C. Computational Efficiency and Convergence Analysis

A deployment-ready attention mechanism must balance representational enhancements with computational efficiency. This subsection quantitatively evaluates the resource efficiency profiles and optimization dynamics of our proposed training-exclusive KA mechanism against conventional plug-and-play attention modules.

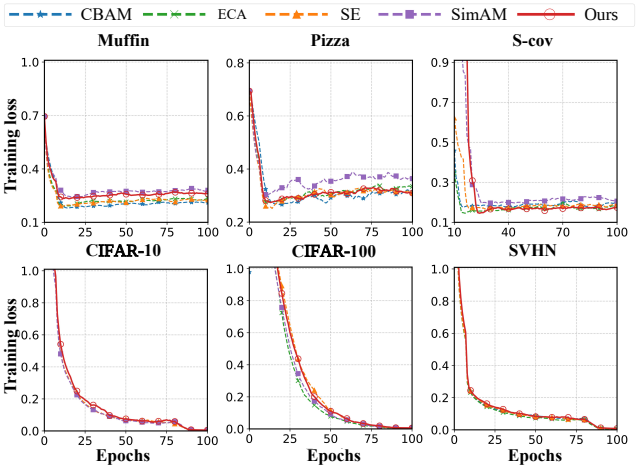


Fig. 4: Variations of loss during training with KA and attention baseline methods in ResNet-18.

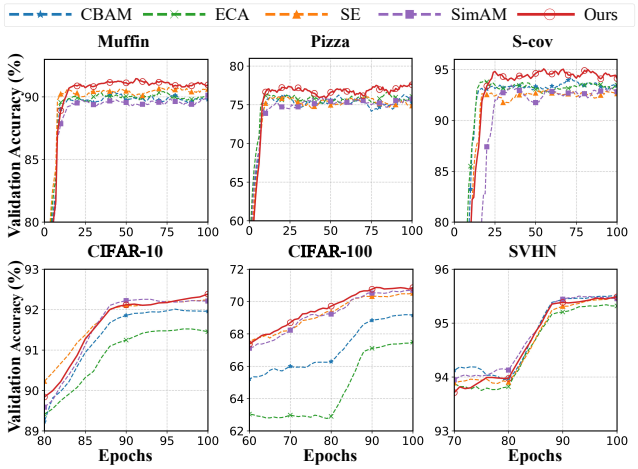


Fig. 5: Variations of accuracy during training with KA and attention baseline methods in ResNet-18.

Analysis on Efficiency. As detailed in Table IV, existing attention mechanisms introduce severe deployment tradeoffs. Parameterized modules like CBAM incur a heavy memory footprint while halving inference throughput (e.g., dropping to 45.85 FPS on ResNet-50). In contrast, the non-parameterized SimAM entails a roughly 30% speed penalty, stemming from its runtime pixel-wise optimization bottleneck. Conversely, our training-exclusive KA introduces strictly zero parameter or FLOP increments (i.e., $\Delta\text{Params} = 0$). Consequently, KA flawlessly preserves the maximum native inference speed across all backbone configurations, establishing optimal computational efficiency for practical real-time systems.

Observation on Optimization Behavior. As delineated in Figs. 4 and 5, KA consistently achieves top-tier validation accuracy on ResNet-18 while maintaining robust convergence. Notably, on the fine-grained *Muffin* and *Pizza* datasets, KA yields a marginally higher asymptotic training loss yet delivers superior validation performance. This counter-intuitive profile

TABLE V: Ablation study on the core components of our proposed method using ResNet-18 as the backbone.

Variant	Dataset						Avg.
	Muffin	Pizza	S-cov	CIFAR-10	CIFAR-100	SVHN	
Vanilla	89.02	77.32	67.91	92.74	71.00	95.76	82.29
w/ Init.	89.27	77.56	71.01	92.76	71.09	95.83	$82.92^{\uparrow 0.63}$
w/ α	89.53	78.69	73.08	92.87	71.61	96.01	$83.63^{\uparrow 1.34}$
Ours	**89.78**	**78.83**	**73.50**	**92.92**	**71.94**	**96.12**	$\mathbf{83.84}^{\uparrow 1.55}$

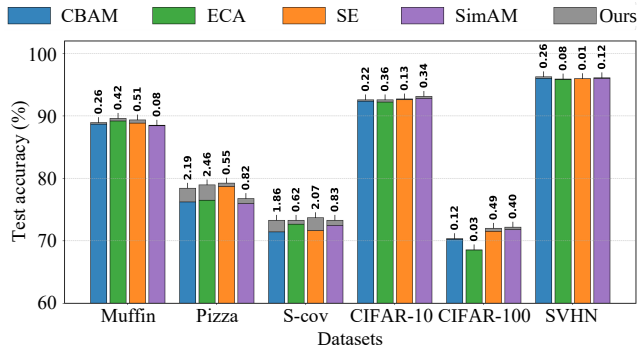


Fig. 6: Test accuracy (%) on six datasets with ResNet-18 using KA combined with existing attention modules. KA consistently improves performance across all datasets.

signifies a potent implicit regularization effect rather than an optimization bottleneck. By binding weight updates to a kernel-driven convex formulation, KA structurally penalizes the over-minimization of empirical risk on localized artifacts, effectively steering the trajectory away from sharp, overfitted regimes and toward flatter local minima with enhanced generalization capabilities.

D. Ablation Study

Table V evaluates the individual and joint contributions of our initialization scheme (*Init.*) and the kernel-machine-derived attention scaling factor α on ResNet-18. Both components deliver independent improvements over the vanilla baseline. Specifically, standalone *Init.* stabilizes the optimization trajectory, whereas the standalone kernel attention factor α yields a major performance boost, raising the accuracy on the *S-cov* dataset from 67.91% to 73.08%. Crucially, the joint integration of both mechanisms yields the optimal performance, achieving a peak average accuracy of 83.84%—a net improvement of 1.55%. This mutual reinforcement demonstrates a powerful architectural synergy in unlocking latent representation capacity.

E. Compatibility with Existing Attention Mechanisms

To verify its orthogonal compatibility, we incorporate KA into various attention-augmented ResNet-18 variants. Because KA is training-exclusive, this integration requires zero modification to the underlying inference architecture or deployment pipeline. As shown in Figure 6, KA consistently amplifies classification performance across all six datasets. Notably,

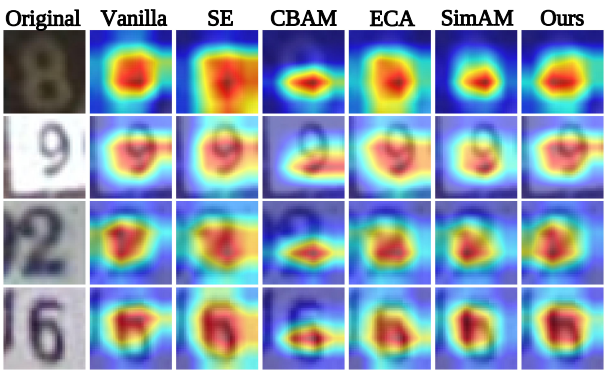


Fig. 7: Qualitative comparison of Grad-CAM activation maps on the *SVHN* dataset. The proposed KA captures more precise, morphology-aware object localization compared to baseline mechanisms.

ECA+KA yields a 2.46% accuracy boost on the *Pizza* dataset, while SE+KA improves the challenging *S-cov* by 2.07%. These results demonstrate that KA serves as an efficient plug-and-play module that enhances feature expressiveness without introducing extra parameters or sacrificing wall-clock throughput.

F. Feature Visualization via Heatmaps

Figure 7 presents Grad-CAM activation maps on the *SVHN* dataset. The baseline exhibits diffuse attention vulnerable to adjacent digits, whereas SE, ECA, and SimAM yield structurally imprecise boundaries, and CBAM suffers from severe spatial over-compression. Conversely, our kernel-derived KA generates concentrated, morphology-aware heatmaps that tightly conform to target contours. This precise semantic grounding validates that KA enhances feature discrimination without parameter overhead.

VI. Conclusion

This paper alleviates the architectural and computational constraints of conventional attention mechanisms in CNNs, which typically rely on parameterized sub-networks or heuristic pooling operations. To resolve these limitations, we have proposed Kernel Attention (KA), a parameter-free mechanism designed to mine input-inherent context. By leveraging convex optimization principles within a statistical kernel learning framework, KA computes non-parametric weights to modulate feature representations while preserving the native structure of the base CNN. Driven by a mathematically formulated initialization strategy, the proposed method eliminates the structural alterations and optimization complexities inherent in existing formulations. Empirical evaluations across multiple benchmarks demonstrate that KA enhances representation performance with zero additional parameters and negligible computational overhead. Furthermore, KA can be integrated into existing attention frameworks to provide complementary performance improvements.

ACKNOWLEDGMENT

This research was funded by the National Natural Science Foundation of China (Grant No. 62376099), the Postdoctoral Fellowship Program of CPSF (Grant No. GZC20251042), the China Postdoctoral Science Foundation (Grant No. 2024M760785), and the Key Research Project Plan of Higher Education Institutions in Henan (Grant No. 26A520002).

REFERENCES

[1] A. Dosovitskiy, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[3] M. Bojarski, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.

[4] J. Redmon, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[5] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3431–3440.

[6] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," in *International Conference on Learning Representations (ICLR)*, 2016. [Online]. Available: https://arxiv.org/abs/1511.07122

[7] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.

[8] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "Cbam: Convolutional block attention module," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 3–19.

[9] Z. Gao, J. Xie, Q. Wang, and P. Li, "Global second-order pooling convolutional networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3024–3033.

[10] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are rnns: Fast autoregressive transformers with linear attention," in *Proceedings of International conference on machine learning (ICML)*. PMLR, 2020, pp. 5156–5165.

[11] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, Ł. Kaiser, D. Belanger, A. Colwell, and A. Weller, "Rethinking attention with performers," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[12] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 20, 2007.

[13] A. G. Wilson, Z. Hu, R. Salakhutdinov, and E. P. Xing, "Deep kernel learning," in *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2016, pp. 370–378.

[14] J. Mairal, P. Koniusz, Z. Harchaoui, and C. Schmid, "Convolutional kernel networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 2627–2635.

[15] J. Mairal, "End-to-end kernel learning with supervised convolutional kernel networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016, pp. 1399–1407.

[16] Y. Cui, F. Zhou, Y. Lin, S. Belongie, and B. Hariharan, "Kernel pooling for convolutional neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2941–2950.

[17] Y. Gao, O. Beijbom, N. Zhang, and T. Darrell, "Compact bilinear pooling," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 317–326.

[18] Y. Xiong, Z. Zeng, R. Chakraborty, and V. Singh, "Nyströmformer: A nyström-based algorithm for approximating self-attention," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2021, pp. 1414–1422.

[19] H. Larochelle and G. E. Hinton, "Learning to combine foveal glimpses with a third-order boltzmann machine," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 23, 2010.

[20] V. Mnih, N. Heess, A. Graves *et al.*, "Recurrent models of visual attention," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014.

[21] J. Park, "Bam: Bottleneck attention module," *arXiv preprint arXiv:1807.06514*, 2018.

[22] X. Li, W. Wang, X. Hu, and J. Yang, "Selective kernel networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 510–519.

[23] H. Zhang, C. Wu, Z. Zhang, Y. Zhu, H. Lin, Z. Zhang, Y. Sun, T. He, J. Mueller, R. Manmatha *et al.*, "Resnest: Split-attention networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 2736–2746.

[24] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "Eca-net: Efficient channel attention for deep convolutional neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 534–11 542.

[25] Z. Qin, P. Zhang, F. Wu, and X. Li, "Fcanet: Frequency channel attention networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 783–792.

[26] H. Zhang, K. Zu, J. Lu, Y. Zou, and D. Meng, "Epsanet: An efficient pyramid squeeze attention block on convolutional neural network," in *Proceedings of the Asian Conference on Computer Vision (ACCV)*, 2022, pp. 1161–1177.

[27] X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7794–7803.

[28] Y. Chen, Y. Kalantidis, J. Li, S. Yan, and J. Feng, "Aˆ 2-nets: Double attention networks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[29] Y. Cao, J. Xu, S. Lin, F. Wei, and H. Hu, "Gcnet: Non-local networks meet squeeze-excitation networks and beyond," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) workshops*, 2019.

[30] L. Yang, R.-Y. Zhang, L. Li, and X. Xie, "Simam: A simple, parameter-free attention module for convolutional neural networks," in *International Conference on Machine Learning (ICML)*. PMLR, 2021, pp. 11 863–11 874.

[31] S. S. Keerthi and C.-J. Lin, "Asymptotic behaviors of support vector machines with gaussian kernel," *Neural Computation*, vol. 15, no. 7, pp. 1667–1689, 2003.

[32] T. Hofmann, B. Schölkopf, and A. J. Smola, "Kernel methods in machine learning," *The Annals of Statistics*, vol. 36, no. 3, pp. 1171–1220, 2008.

[33] B. Schölkopf, R. Herbrich, and A. J. Smola, "A generalized representer theorem," in *International Conference on Computational Learning Theory*. Springer, 2001, pp. 416–426.

[34] A. J. Smola and B. Schölkopf, *Learning with Kernels*. Citeseer, 1998, vol. 4.

[35] T. M. Nguyen, T. Nguyen, N. Ho, A. L. Bertozzi, R. G. Baraniuk, and S. J. Osher, "A primal-dual framework for transformers and neural networks," *arXiv preprint arXiv:2406.13781*, 2024.

[36] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[37] D. A. Tarzanagh, Y. Li, C. Thrampoulidis, and S. Oymak, "Transformers as support vector machines," *arXiv preprint arXiv:2308.16898*, 2023.

[38] T. Lin, "Focal loss for dense object detection," *arXiv preprint arXiv:1708.02002*, 2017.

[39] A. Shrivastava, A. Gupta, and R. Girshick, "Training region-based object detectors with online hard example mining," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 761–769.

[40] Kaggle, "Muffin-vs-chihuahua," https://www.kaggle.com/datasets/samuelcortinhas/muffin-vs-chihuahua-image-classification, 2022.

[41] ——, "Pizza-not-pizza," https://www.kaggle.com/datasets/carlosrunner/pizza-not-pizza, 2022.

[42] E. Soares, P. Angelov, S. Biaso, M. Higa Froes, and D. Kanda Abe, "Sars-cov-2 ct-scan dataset: A large dataset of real patients ct scans for sars-cov-2 identification," *medRxiv*, 2020. [Online]. Available: https://www.medrxiv.org/content/early/2020/05/14/2020.04.24.20078584

[43] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[44] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, A. Y. Ng *et al.*, "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, vol. 2011, no. 2. Granada, 2011, p. 4.